

ALGORITHM:- LOGISTIC REGRESSION

Logistic Regression is a supervised machine learning algorithm used for classification — most often for binary classification (two classes), but it can also be extended to multi-class problems.

Even though it's called “regression,” it is actually a classification algorithm. The name comes from the fact that it uses a linear regression model as input but applies a logistic (sigmoid) function to produce probabilities.

Algorithm Type & Objective

- **Algorithm Type:**

- Linear Model – Models a linear relationship between review text features and the log-odds of each rating class.
- Supervised Learning – Uses historical reviews with known ratings as labeled data.
- Probabilistic Classifier – Outputs probabilities for each possible rating (e.g., 1–5 stars).

- **Objective:**

- Predict the most likely star rating (1 to 5) for a given review.
- Learn the weights for words/phrases that best separate ratings (e.g., “excellent” → high rating, “terrible” → low rating).

Mathematical Intuition

1. Feature Representation

- Represent each input text as a numerical feature vector x (e.g., TF-IDF, Bag of Words).
- Each dimension of x corresponds to a feature with a numeric value.

$$z = w \cdot x + b$$

2. Linear Combination

Where:

- w = weight vector (learned parameters)
- x = input feature vector
- b = bias term

3. Probability Mapping

- Binary classification uses the sigmoid function:

$$P(y = 1|x) = \frac{1}{1 + e^{-z}}$$

- Multi-class classification uses the softmax function:

$$P(y = k|x) = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}$$

4. Decision Rule

- For binary: Assign class 1 if $P(y=1|x) > 0.5$, else class 0.
- For multi-class: Assign the class with the highest probability.

5. Training Objective

- Minimize the cross-entropy loss between predicted probabilities and actual labels:
Binary:

$$L = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

Multi-class:

$$L = -\frac{1}{N} \sum_{i=1}^N \sum_{k=1}^K y_{i,k} \log(\hat{y}_{i,k})$$

6. Optimization

- Update parameters w and b using gradient descent or other optimization solvers:

$$\begin{aligned}w &:= w - \eta \frac{\partial L}{\partial w} \\b &:= b - \eta \frac{\partial L}{\partial b}\end{aligned}$$

Key Hyperparameters

Hyperparameter	What It Controls	Default	Tuning Tips
C	Inverse of regularization strength	1.0	Lower for stronger regularization (less overfitting).
penalty	Regularization type	l2	l1 can help remove irrelevant words/features.
solver	Optimization algorithm	lbfgs	saga is better for large datasets & sparse features.
max_iter	Max training iterations	100	Increase if you get convergence warnings.
multi_class	Multi-class handling	auto	Use multinomial for better multi-star accuracy.

Strengths & Limitations

Strengths

- Works very well with TF-IDF features from review text.
- Interpretable: You can see which words push ratings up or down.
- Fast to train, even with thousands of reviews.
- Regularization helps avoid overfitting with large vocabularies.

Limitations

- Assumes linear separation between ratings in feature space.
- May confuse ratings close to each other (e.g., 3-star vs 4-star).
- May miss complex patterns like sarcasm or mixed sentiment.
- Needs a decently large dataset for multi-class ratings.

When to Use / When Not to Use

Use When:

- You want a fast, interpretable baseline for predicting ratings.
- You have a large set of labeled reviews with ratings.
- You are working with high-dimensional sparse text features (TF-IDF).

Avoid When:

- You need state-of-the-art accuracy for nuanced review understanding (transformers like BERT may outperform).
- You have a small dataset and complex patterns in ratings.
- You need to model non-linear relationships between text and ratings.